

July 12, 2026

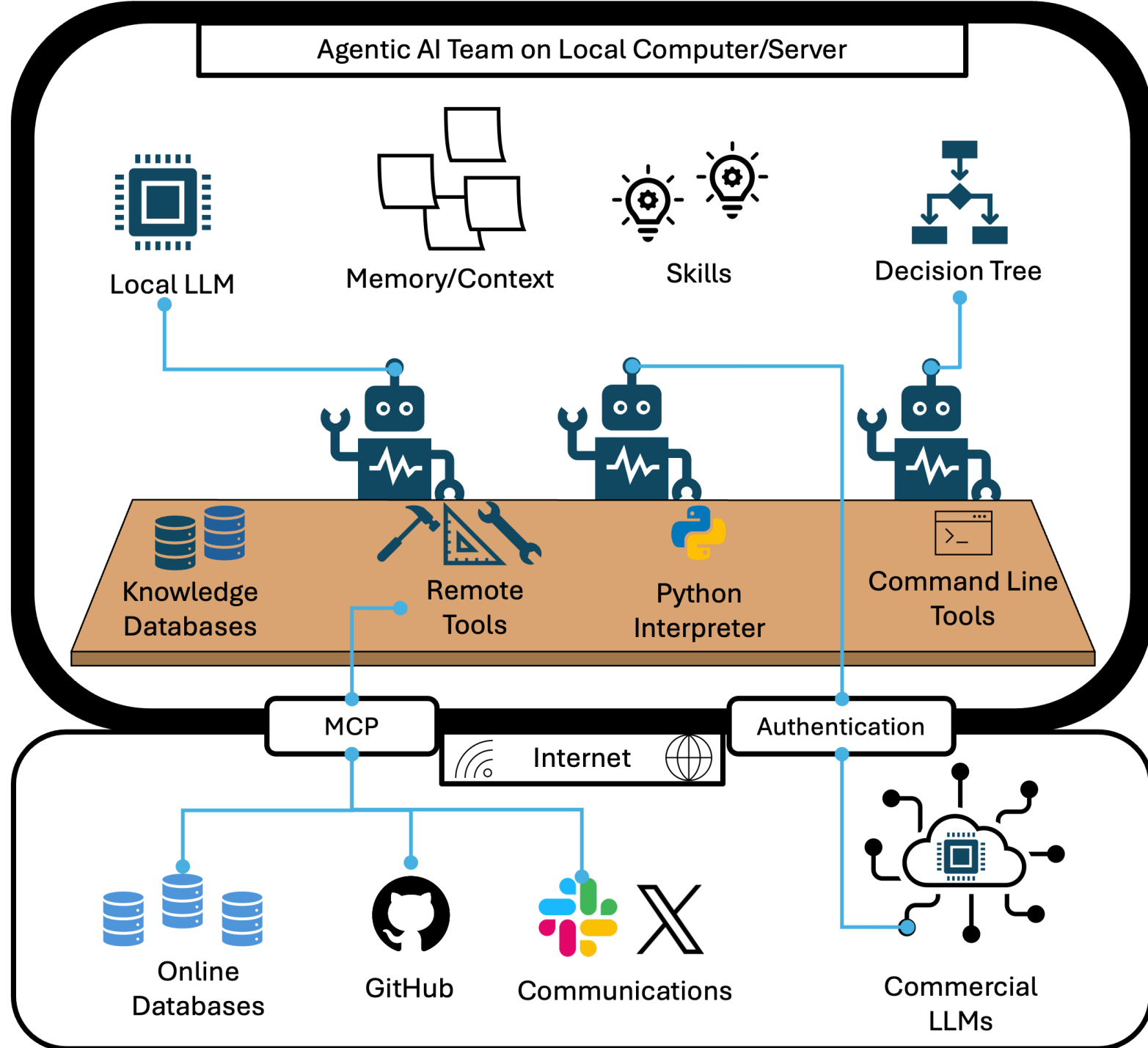
Engineering *In Silico* Teams of Agents

Hands-On Materials Available on GitHub

Check out tutorial Jupyter notebooks: <https://github.com/EpistasisLab/ismb-2026-biomedical-agent-engineering>

Notebook Contents			
#	Notebook	Description	Key Topics
01	GUI Agent Builders	Explore three no-code platforms for building AI agents side-by-side, with a hands-on first agent in each	N8N visual workflows, Flowise RAG pipelines, AutoGen Studio multi-agent teams; comparison table; when to move to code
02	Python Client SDKs	A "with vs. without" walkthrough of eight core agentic concepts across the Anthropic, OpenAI, and LangChain SDKs	Model tiers, system prompts, tool/function calling, in-context vs. checkpointer memory, streaming, structured output with Pydantic
03	MCP Agents	Bind Model Context Protocol servers to LLM agents so tools are discovered at runtime; includes building a custom PubMed MCP server	MCP architecture (stdio vs. HTTP), FastMCP server authoring, three binding patterns: Anthropic beta API, manual <code>ClientSession</code> , LangChain <code>MultiServerMCPClient</code>
04	ReAct Agents	Implement the ReAct loop (Thought → Action → Observation) for multi-step biomedical tool use, both with LangGraph's prebuilt agent and a raw SDK implementation	<code>create_react_agent</code> , <code>@tool</code> with Pydantic schemas, streaming intermediate steps, parallel tool calls, manual Anthropic tool-use message cycle
05	Reflection — PGx Use Case	Apply the reflection pattern to pharmacogenomics drug dosing: generate a draft, retrieve CPIC guidelines, then refine — implemented in both LangChain LCEL and direct Anthropic SDK	CPIC diplotype → phenotype → recommendation pipeline, LCEL chains, V1→V2 diff with <code>redlines</code> , batch evaluation across 19 genes
06	Multi-Agent Orchestration	Build a three-agent supervisor pipeline (Literature Summary → Critic → Supervisor) for automated scientific paper review, in three different SDKs	OpenAI Agents SDK <code>as_tool()</code> , Anthropic SDK tool-use loop, LangGraph <code>StateGraph</code> ; SDK tradeoff comparison table

The Anatomy of AI Agents

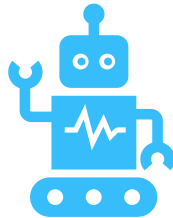


The Anatomy of AI Agents



Reasoning Engine

Large Language Models
Decision Trees



AI Agent
Supervisor
Geneticist Critic
ML Engineer
Reporter



Agency

Local Tools
Model Context Protocol (MCP) for
Access to Remote Tools and Services



Domain Expertise

Prompt Engineering
Retrieval-Augmented Generation (RAG)



Progress Management

Memory
Context

Opportunities by Agent Engineering Stack

1

Graphic User Interface (GUI) Tools

2

Python Software Development Kits (SDKs)

3

Command-Line Interface (CLI) Tools

Graphic User Interface (GUI) Tools

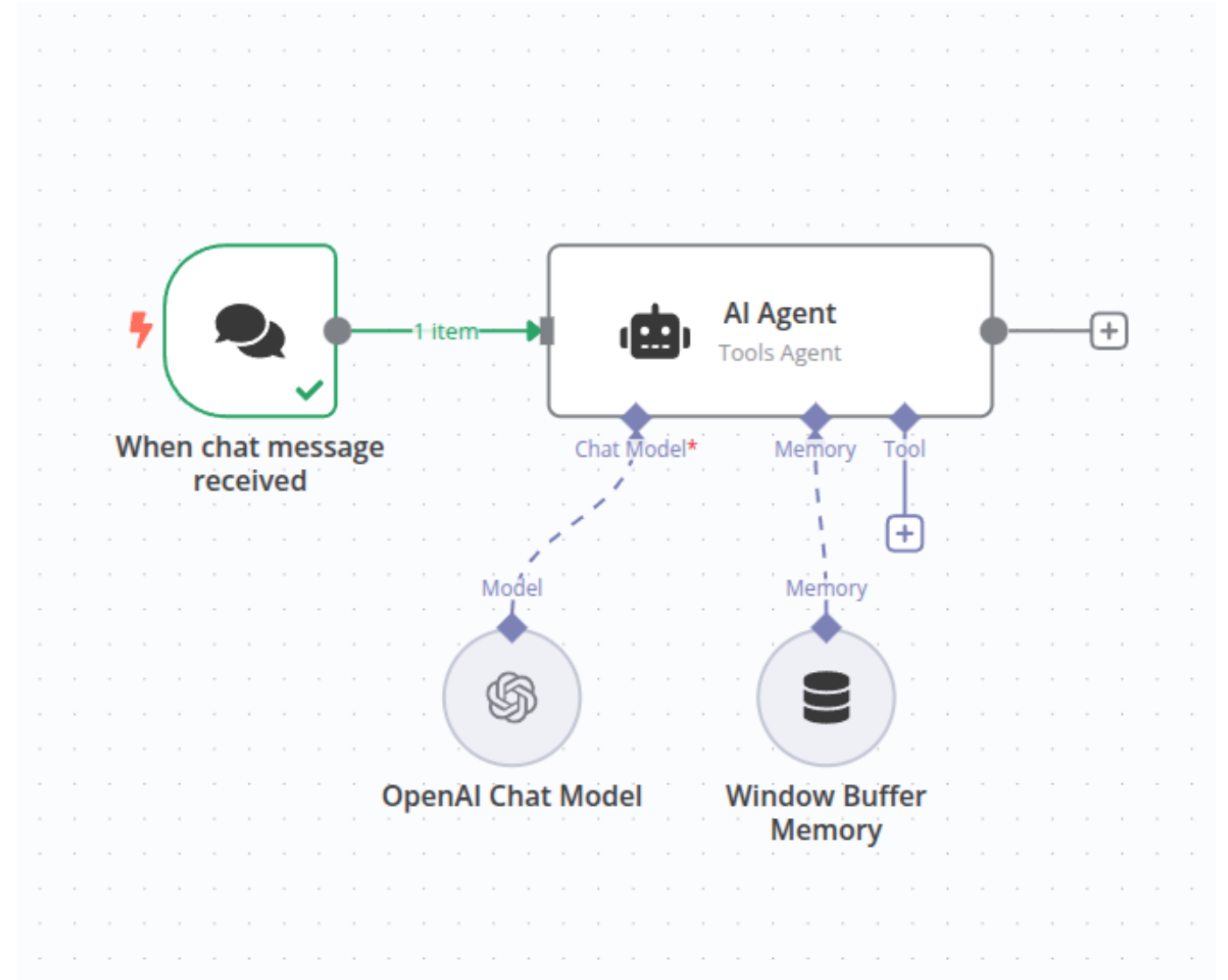
Why Start With a GUI?

No-code Agentic AI Prototyping

Visualized engineering

Nodes = Agents, tools, conditions...

Edges = Data flow, memory, context

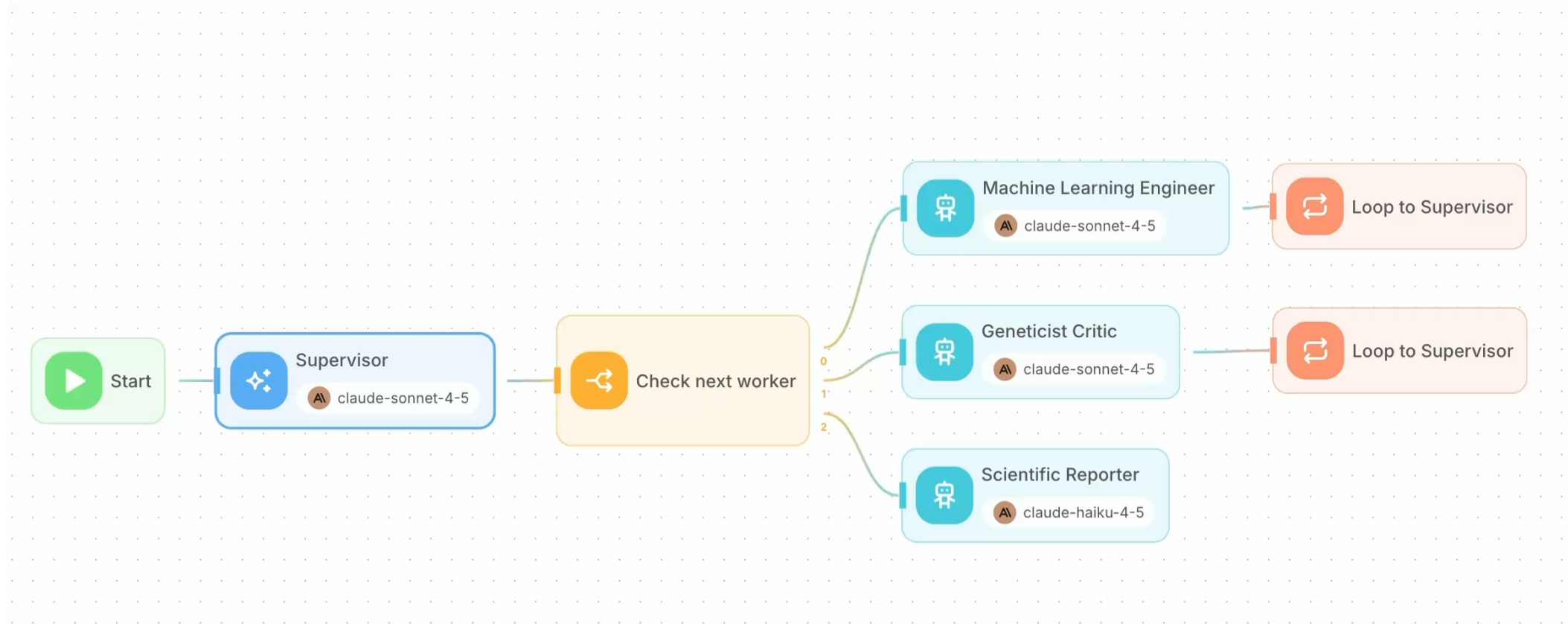


Demo: Building a Multi-Agent Orchestration System

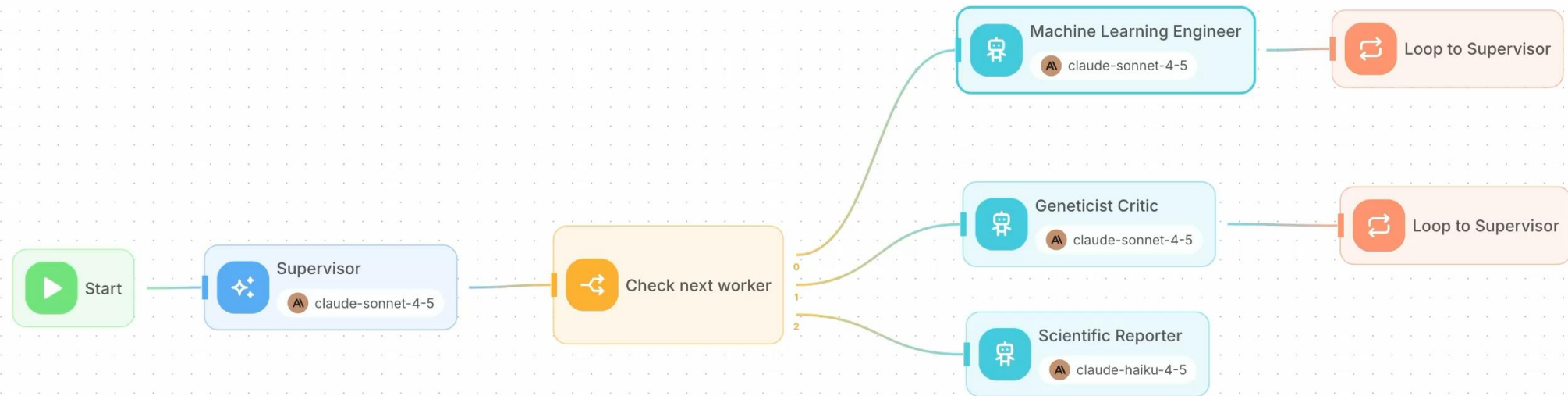
Drag and drop system components (agents, trigger, routing decisions) in a visually intuitive way.



Configuring AI Agents in Action

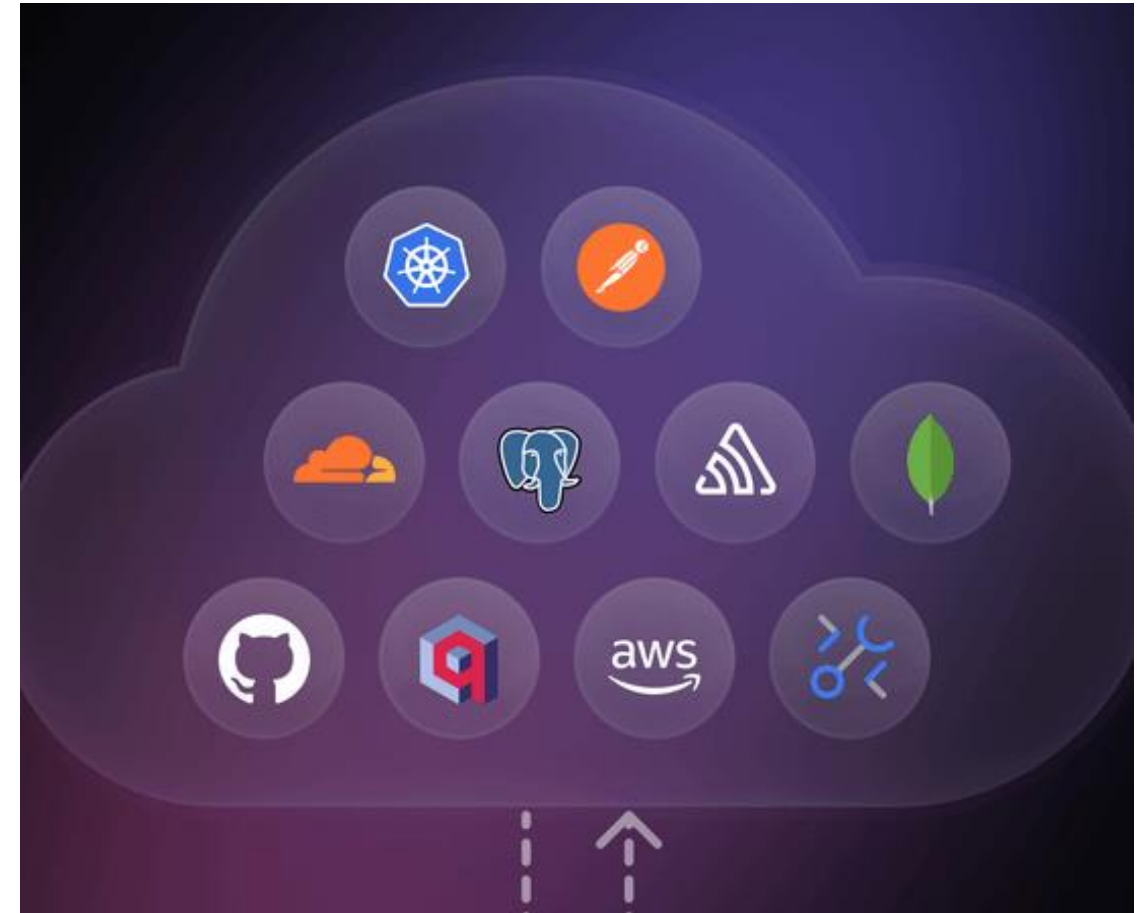


Enhance Domain Expertise: Prompt, MCP, RAG

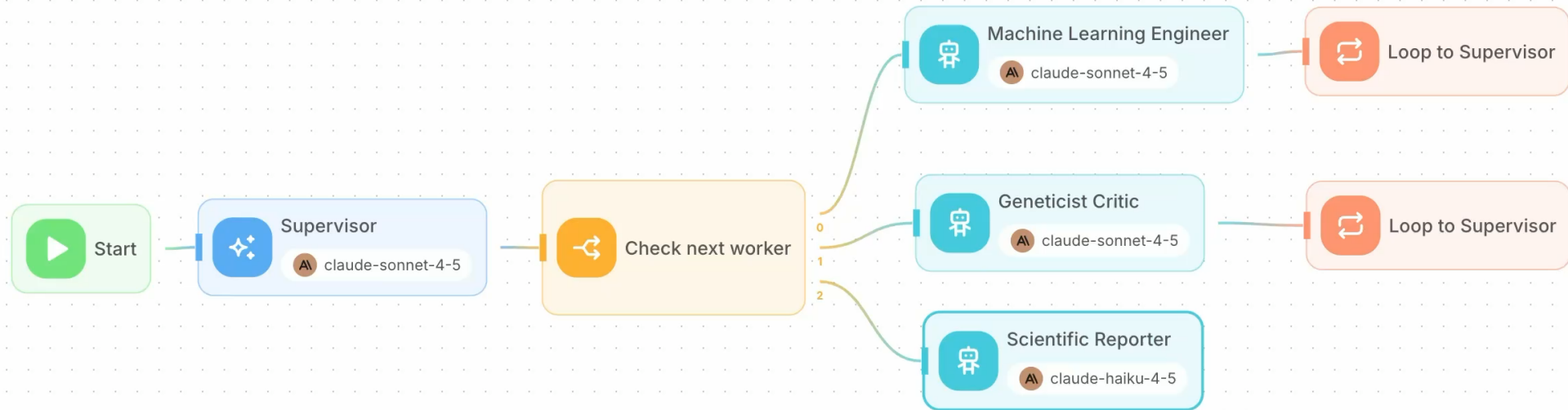


GUI-Based Agent Engineering Platforms Come With Support for Various MCP tools

- **Support for MCP infrastructure.**
- **Credentials are usually the only requirements.**
- **Pre-configured, common MCP tools:**
 - SQL databases, e.g., PostgreSQL
 - Cloud services (AWS, Azure, Google Cloud)
 - GitHub
 - Web Search tools
 - Notion



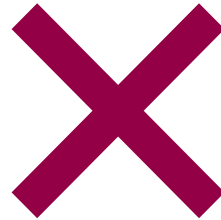
Executing the Whole Multi-Agent Orchestration Workflow



Takeaways – GUI Platforms for Agent Engineering



- **Rapid prototyping**
- **Non-coding**
- **Webhook pipelines**
- **Portable via JSON exports**



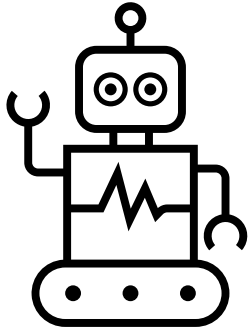
- **Version control**
- **Flexibility for complex logic**
- **Tied by available tools and services provided by the platform**



- **Available Platforms**
- N8N
- Flowise AI
- AutoGen Studio

Python SDKs

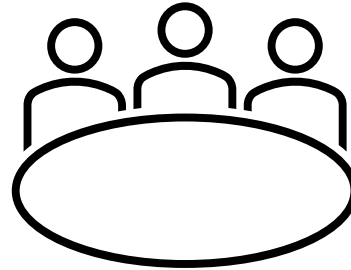
Agentic SDK Landscape



LLM Client SDKs

Handles API interactions with LLM providers.

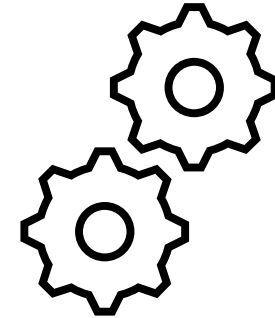
For example, Anthropic Client SDKs, OpenAI SDKs, and LangChain.



Agents SDKs

Supports dynamic agent configurations, including agent personas, roles, inter-agent coordination patterns.

For example, Claude Agent SDK, OpenAI Agents SDK, and CrewAI.



Other SDKs

LangGraph: Graph-based orchestration that allows deterministic hand-offs among agents, tools, and humans.

Configuring LLM Agents Using Python Client SDKs

Python client SDKs from different providers have distinct but similar API structures.

Common agentic AI parameters

- Model selection
- Cost, max context window, tool support, reasoning capabilities
- Conversation state (memory/context)
- Access to chat history
- Model settings
- Max tokens, temperature, stop sequence, streaming
- Tool/Function calling settings
- Controls whether and when the agent may or must call a tool.
- User input
- Conversation, task context
- **System/Assistant/Human messages**

```
question = "Name three key differences between DNA and RNA in one sentence each."

# Anthropic: direct API call with different model parameter
anthropic_client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=256,
    messages=[{"role": "user", "content": question}],
)

# OpenAI: similar pattern but different API parameters
openai_client.chat.completions.create(
    model="gpt-5.4-mini",
    max_completion_tokens=256,
    messages=[{"role": "user", "content": question}],
)

# OpenAI Responses API
openai_client.responses.create(
    model="gpt-5.4-mini",
    input="What is PCR used for?",
)

# LangChain: harmonized interface abstracts away API differences
model = ChatAnthropic(model="claude-haiku-4-5-20251001", max_tokens=256)
model = ChatOpenAI(model="gpt-5.4-mini", max_tokens=256)
model.invoke(question)
```


Message Types Affect Agent Behaviors.

System message determines an agent's persona, constraints, tone, output format, and human preferences.

System message

- Agents: "how I should behave."

Assistant message

- Agents: "what I have said."

Human message

- Agents: "what I am asked to do now."

```
genomics_system = (  
    "You are a genomics assistant. "  
    "Answer only questions about genes, genomes, sequencing, or molecular biology. "  
    "For any other topic, politely decline and redirect to genomics."  
)  
  
questions = [  
    "What does high GC content mean for PCR primer design?", # in scope  
    "Can you recommend a good restaurant in Boston?",         # out of scope  
    "What is the capital of France?",                           # out of scope  
)  
  
for q in questions:  
    response = anthropic_client.messages.create(  
        model="claude-sonnet-4-6",  
        max_tokens=100,  
        system=genomics_system,  
        messages=[{"role": "user", "content": q}],  
    )  
    print(f"Q: {q}")  
    print(f"A: {response.content[0].text}\n")
```

System Prompt Shapes Agent Behaviors

System prompt restricts the agent to genomics questions.

```
genomics_system = (  
    "You are a genomics assistant. "  
    "Answer only questions about genes, genomes, sequencing, or molecular biology. "  
    "For any other topic, politely decline and redirect to genomics."  
)
```

Q: Can you recommend a good restaurant in Boston?
(out of scope)

```
A: I appreciate you asking, but restaurant recommendations are outside my area  
of expertise! I focus specifically on genomics, genes, sequencing, and  
molecular biology topics.
```

Q: What does high GC content mean for PCR primer
design? (in-scope)

```
A: High GC content has several important implications for primer design:  
- G-C base pairs form 3 hydrogen bonds (vs. 2 for A-T pairs)  
- This makes GC-rich sequences more thermally stable  
- Directly affects melting temperature (Tm)
```

Q: What is the capital of France?
(out of scope)

```
A: That's a geography question, which is outside my area of expertise! I'm  
specialized in genomics, molecular biology, and sequencing topics.
```

Remote Reasoning, Local Tool Use

Agents do not run tools inside the LLM.

The LLM only decides which tool to call and returns a structured function call.

Tool definitions are sent with the prompt.

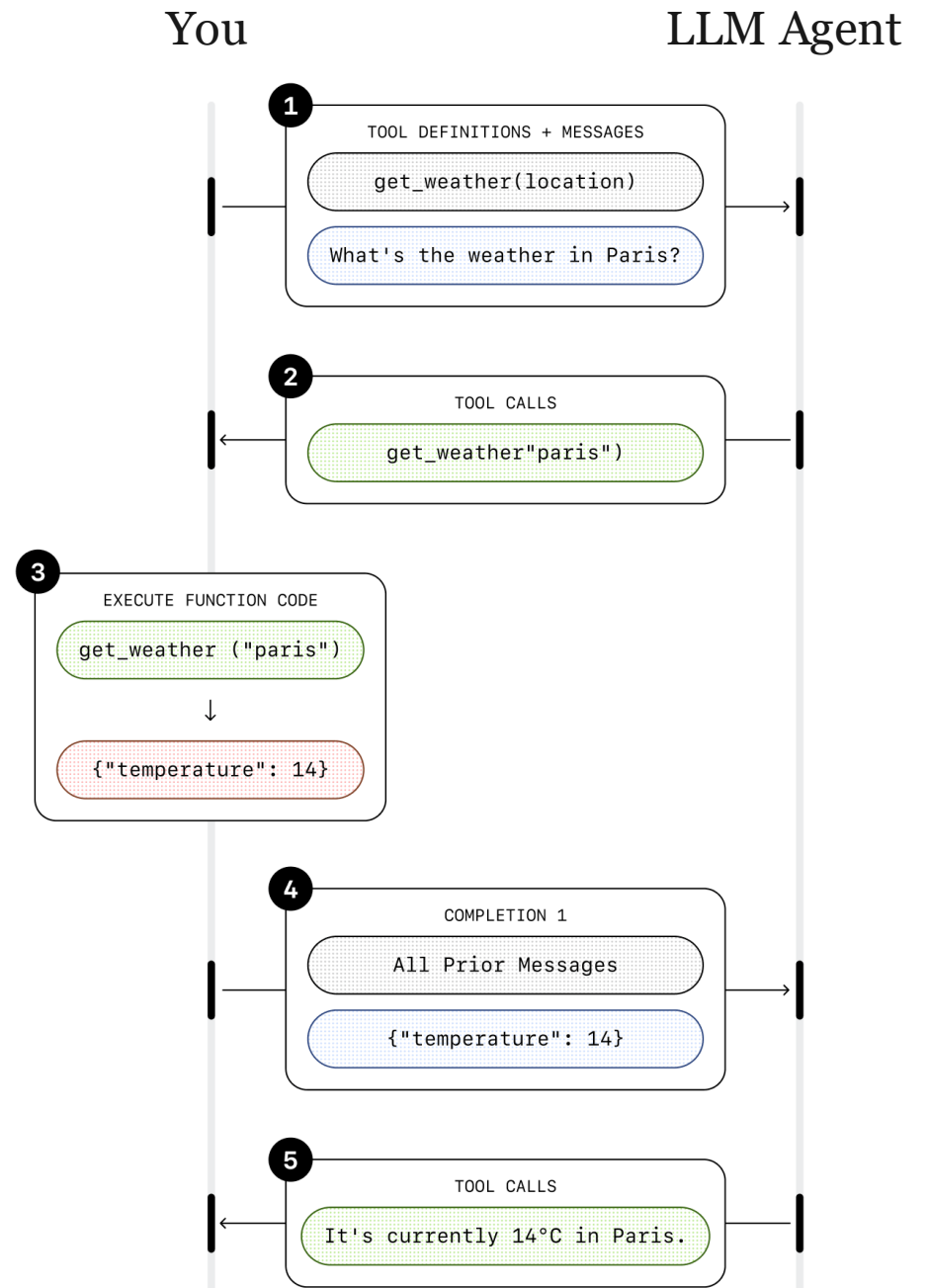
Each tool has a name, a description, and an input schema, like `get_weather(location)`.

The client code executes the tool locally.

Your app, server, or framework runs the actual Python/API/database call.

Tool results are sent back to the LLM.

The model uses the returned data to continue reasoning or produce the final answer.



Extend Agents Capabilities With MCPs

MCP standardizes how agents discover and call external tools, resources, and services.

The LLM still does not execute the tools; it requests a tool call to the right MCP server.

The MCP server returns the tool call result to the model.

```
WITH Anthropic beta MCP (tools from biocontext.ai):
Here are the tools I have available, grouped by category:

---

### 🧬 Protein & Gene Information
- **get_uniprot_id_by_protein_symbol** - Convert protein/gene name to UniProt accession ID
- **get_uniprot_protein_info** - Retrieve detailed protein info from UniProt
- **get_alphafold_info_by_protein_symbol** - Fetch AlphaFold structure predictions
- **get_ensembl_id_from_gene_symbol** - Convert gene symbol to Ensembl ID
- **get_protein_domains** - Get protein domain architecture via InterPro
- **get_human_protein_atlas_info** - Expression, localization, and pathology data

---

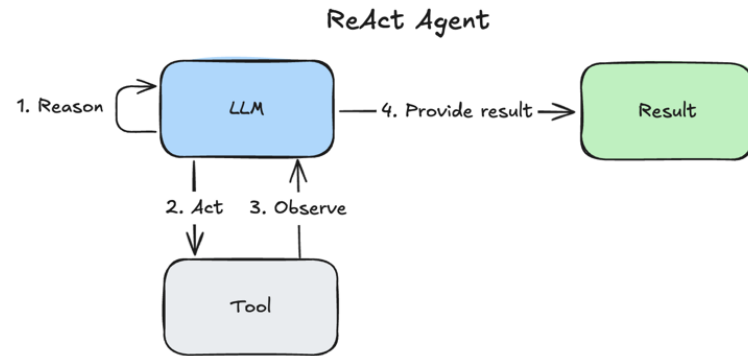
### 🧪 Proteomics & Structural Biology
- **get_pride_project** - Detailed metadata for a PRIDE proteomics project
- **search_pride_projects** - Search PRIDE database for MS proteomics datasets
- **search_pride_proteins** - Find proteins identified in a PRIDE project

---

### 🌐 Protein Interactions & Pathways
- **get_string_id** - Map protein names to STRING database IDs
- **get_string_interactions** - Retrieve protein-protein interactions from STRING
- **get_string_network_image** - Generate PPI network visualization from STRING
- **get_string_similarity_scores** - Homology similarity scores between proteins
- **get_reactome_info_by_identifier** - Retrieve Reactome pathways for an identifier
```

Infrastructure Patterns to Improve Agentic Responses

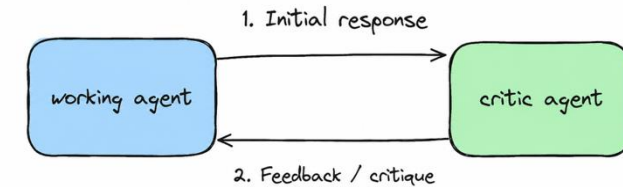
ReAct (reasoning and acting)



Agent response with ReAct

```
[Human] What is 37 times 42?  
[AI] tool_call: multiply({'a': 37, 'b': 42})  
[Tool] 1554.0  
[AI] 37 times 42 is **1,554**.
```

Reflection



Agent response with reflection

Step 3 — Reflection Feedback

The initial recommendation correctly identifies the *4/*4 diplotype as a poor metabolizer phenotype, accurately explains the mechanism of reduced morphine formation, and appropriately recommends avoiding codeine with suitable alternatives. However, it omits the CPIC-specific warning against tramadol, which is also contraindicated in CYP2D6 poor metabolizers and is an important clinical consideration.

Multi-Agent Orchestration: Divide the Task Across Specialists

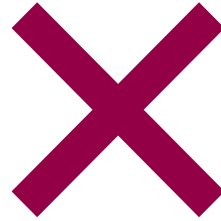
- Each agent has a role: supervisor, summarizer, or critic.
- The supervisor controls the workflow, deciding the next agent and organizing the output from the previous agent to the next.
- Orchestration can be fixed or dynamic: a predefined pipeline, a conditional graph, or a supervisor that chooses the next step.
- Agents can be exposed as tools: one agent can call another agent like a function.
- Best for complex tasks, such as multi-step planning or iterative improvement.

```
supervisor_openai = Agent(  
    name="supervisor",  
    instructions="""You are a senior scientific editor coordinating a structured  
    paper review.  
  
    Given a PMID, follow these steps IN ORDER:  
    1. Call `literature_summary_agent` with the PMID – it will fetch the paper and  
    return a structured summary.  
    2. Call `critic_agent` with the summary text returned in step 1.  
    3. Synthesize both responses into a final structured review report with four clearly  
    labeled sections:  
        ## Significance  
        ## Novelty  
        ## Technical Soundness  
        ## Major Critiques  
  
    End with a one-paragraph Overall Assessment."""  
)  
  
tools=[  
    summary_agent.as_tool(  
        tool_name="literature_summary_agent",  
        tool_description="Fetches the paper for a given PMID and summarizes its  
        significance, novelty, and technical soundness.",  
    ),  
    critic_agent.as_tool(  
        tool_name="critic_agent",  
        tool_description="Critically reviews a literature summary as an  
        independent peer reviewer. Pass the summary text from  
        literature_summary_agent.",  
    ),  
]  
model="gpt-4o-mini",  
)  
  
result = await Runner.run(  
    supervisor_openai,  
    f'Review the paper with PMID {PMID}',  
    max_turns=25  
)
```

Takeaways – Python Agents SDK



- **Reproducibility**
- **Testing**
- **Version control**
- **Integration with existing Python projects**
- **Fine-grained control**



- **Python proficiency**
- **Complex configuration of agentic infrastructure**



- **Available Toolkits**
- OpenAI
- Anthropic
- Ollama
- LangChain/LangGraph
- CrewAI

Command-Line Interface (CLI) Tools

Coding Agents Are Moving Into the Terminal And IDE

CLI coding agents turn LLMs from code-completion tools into terminal-native software engineering collaborators.

- Combine LLM reasoning with local tool use and project context.
- Read, edit, run, and debug code within a project.
- shell commands, tests, linters, file edits, Git, package managers, and MCP tools.
- Humans' roles shift.
- From biomedical programmer to agent engineer and supervisor.
- LLM providers are converging on the same pattern.
- Agent loop + codebase context + tool execution + human approval.
- Python client SDKs + CLI coding agents + IDE



CLAUDE.md: Context Provides Better Performance

```
# Core coding principles
- Prioritize readability, simplicity, and maintainability for a computational biologist.
- Avoid overengineering, specifically, avoid new parameters that does not help prototype.
- Avoid ambiguous variable names. Use full or short names.
- Keep project architecture simple.
- Follow Python official style guide (PEP 8).
- Update reference documentation such as README.md, ./*.md, .claude/CLAUDE.md and .claude/*.md after implementing new features.
- Never commit secrets.
- When in doubt, ask for clarification before proceeding.
```

```
# CLAUDE.md
Project-specific guidance on top of ~/.claude/CLAUDE.md

## Goal

BaseAgent is a multi-agent AI system that autonomously develops disease-specific knowledge graphs. Given a user prompt specifying a disease scope (e.g., "Alzheimer's disease", "tuberculosis"), a template codebase, and a reference ontology, BaseAgent produces a `<disease>-KG` -- a Python-based codebase for building a disease-specific, memgraph-compatible, OWL-guided knowledge graph.
```

`~/.claude/CLAUDE.md`

- Global, across projects
- Document common slash commands, constraints, preferences
- Keep it short

`/path/to/project/dir/CLAUDE.md`

- Project-specific instructions
- Share with project collaborators
- Use CLAUDE.local.md for personal settings

Build Custom Agents For Multi-Agent Teams

Use /agents to create custom agents

- Global, across project
- For example, a general critic agent
- ~/.claude/agents/
- Project-wide
- For example, a project code reviewer agent
- /path/to/project/.claude/agents/
- Spawning specialized sub-agents
- Use @ or directly mention the agent's name
- Parallel vs. sequential vs. background agent execution
- Specify in the prompt.

1

2

```
Summarize the latest feature of exporting edge properties in
@template/src/ontology/populator.py and @template/src/export/memgraph_exporter.py.
Ask the @"baseagent-reviewer (agent)" and the general critic agent to evaluate
the soundness and simplicity of the change. Come up with an improvement plan if
necessary.
└─ Read template/src/ontology/populator.py (unchanged)
└─ Read template/src/export/memgraph_exporter.py (unchanged)

I'll read both files first, then run the two review agents in parallel.

Read 2 files (ctrl+o to expand)

Let me check the git diff to understand what changed.

Bash(git diff HEAD template/src/ontology/populator.py
template/src/export/memgraph_exporter.py)
└─ diff --git a/template/src/export/memgraph_exporter.py b/template/src/export
/memgraph_exporter.py
index f5a70a7..c786c42 100644
... +208 lines (ctrl+o to expand)

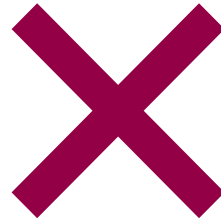
Now I have a full picture of the changes. Let me run both review agents in
parallel.

Running 2 agents... (ctrl+o to expand)
└─ baseagent-reviewer (BaseAgent code review of edge properties feature) · 2 tool
uses · 8.3k tokens
└─ Reading 2 files...
└─ general-critic (General critic review of edge properties feature design) · 0
tool uses
└─ Done
(ctrl+b to run in background)
```

Takeaways – CLI Coding Agents



- **Coding Companions**
- **Day-to-day assistants**
- **Software engineering tasks**
- **Spontaneous interactions**



- **Scaling costs with agent depths**
- **Requiring clear task scopes**
- **No workflow customization**



- **Available Tools**
- Claude Code
- OpenAI Codex
- Google Gemini/Antigravity
- IDEs with built-in agents

Choosing Your Own Agent Engineering Platform

	GUI Platforms	Python SDKs	CLI Coding Agents
Audience	No-code	Developers	Developers
Prototyping	Fast	Low	Medium
Flexibility	Medium	High	Low
Version control	Hard	Native	Native
Use cases	Complex workflows with visual orchestration	Complex, high-performance data pipelines	Daily coding tasks
Learning curve	Medium	High	Low

Next Step: Start With Your First Agent

Check out tutorial Jupyter notebooks: <https://github.com/EpistasisLab/ismb-2026-biomedical-agent-engineering>

Notebook Contents			
#	Notebook	Description	Key Topics
01	GUI Agent Builders	Explore three no-code platforms for building AI agents side-by-side, with a hands-on first agent in each	N8N visual workflows, Flowise RAG pipelines, AutoGen Studio multi-agent teams; comparison table; when to move to code
02	Python Client SDKs	A "with vs. without" walkthrough of eight core agentic concepts across the Anthropic, OpenAI, and LangChain SDKs	Model tiers, system prompts, tool/function calling, in-context vs. checkpointer memory, streaming, structured output with Pydantic
03	MCP Agents	Bind Model Context Protocol servers to LLM agents so tools are discovered at runtime; includes building a custom PubMed MCP server	MCP architecture (stdio vs. HTTP), FastMCP server authoring, three binding patterns: Anthropic beta API, manual <code>ClientSession</code> , LangChain <code>MultiServerMCPClient</code>
04	ReAct Agents	Implement the ReAct loop (Thought → Action → Observation) for multi-step biomedical tool use, both with LangGraph's prebuilt agent and a raw SDK implementation	<code>create_react_agent</code> , <code>@tool</code> with Pydantic schemas, streaming intermediate steps, parallel tool calls, manual Anthropic tool-use message cycle
05	Reflection — PGx Use Case	Apply the reflection pattern to pharmacogenomics drug dosing: generate a draft, retrieve CPIC guidelines, then refine — implemented in both LangChain LCEL and direct Anthropic SDK	CPIC diplotype → phenotype → recommendation pipeline, LCEL chains, V1→V2 diff with <code>redlines</code> , batch evaluation across 19 genes
06	Multi-Agent Orchestration	Build a three-agent supervisor pipeline (Literature Summary → Critic → Supervisor) for automated scientific paper review, in three different SDKs	OpenAI Agents SDK <code>as_tool()</code> , Anthropic SDK tool-use loop, LangGraph <code>StateGraph</code> ; SDK tradeoff comparison table

AI Providers Now Move Into the Biomedical Research Field.

Claude Science

Beta

← Literature review

+ New

Customize

Files

Active

Cross-species scRNA-s...

Cross-species scRNA-seq Integration...

Write a literature review on cross-species single-cell RNA-seq integration. Pull the primary methods papers and recent benchmarks. Output the report as a LaTeX doc and a compiled PDF.

> Ran 4 searches, loaded 2 skills, managed environments, +2 more

10 steps

Dispatching five parallel literature-retrieval tracks — PubMed primary methods, bioRxiv preprints, OpenAlex citation counts, CELLxGENE multi-species atlas inventory, and orthology-free embedding methods.

✓ Dispatching PubMed bioRxiv OpenAlex CELLxGENE sub-agents

142 lines of output

Reviewer · 1 finding

⚠ Warn

PMID 31178118 assigned to both LIGER and Seurat v3 integration in the plan

In the generate_plan PubMed delegation step the agent writes "LIGER (31178118), Seurat v3 integration (31178118)" — the same PMID for two distinct primary methods papers. The same plan's OpenAlex step assigns them different DOIs (Seurat v3 10.1016/j.cell.2019.05.031, LIGER 10.1016/j.cell.2019.05.006), so the plan is internally inconsistent and at least one PMID is wrong. None of the PMIDs or DOIs in msg[10] trace to any in-window tool output — no PubMed/CrossRef/OpenAlex lookup has run (exec-log rows 04449b53, 57baea0a, 254f1378 are the two skill kernel.py auto-loads and the env-filter bash; msg[4] tool_results are skill-catalog listings only).

The agent reads these findings and self-corrects in its next message.

Acknowledged — the plan listed PMID 31178118 for both; the PubMed sub-agent caught the swap and the saved CSV carries the corrected pair (LIGER 31178122, Seurat v3 31178118).

all 5 agents done

Reviewing

Ask anything

+ ⌨

🗣️ ↗️

review.pdf

Cross-species single-cell RNA-seq integration: from one-to-one orthologs to protein-language-model embeddings

Literature synthesis compiled from PubMed, bioRxiv, OpenAlex and CELLxGENE Discover

Papers	Years	Methods	Species pairs	Top-cited	Most recent
24	2018–2025	15	6 benchmarked	Seurat v3 (16,859 cit.)	TranscriptFormer (2025)

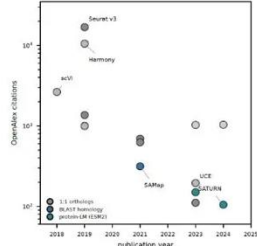


Figure 1: Fifteen integration methods (2018–2024) coloured by orthology strategy. OpenAlex citation counts on log scale. Orthology-subsetting tools (greys) dominate citations because they double as batch-correction methods; the dedicated cross-species tools—SATmap (BLAST homology graph) and SATURN/UCE (ESM2 protein embeddings)—are recent and lower-cited but uniquely span >400 Mya divergence.

autoencoder over a shared gene set, and scANVI [7], scArches [8] and scPool [9] extend it with semi-supervised label transfer and reference-mapping adapters. The scVI evidence lower bound,

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x}, \mathbf{s})} [\log p_\theta(\mathbf{x} | \mathbf{z}, \mathbf{s})] - D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}, \mathbf{s}) \| p(\mathbf{z})). \quad (1)$$

with $\mathbf{s}$ the species/batch indicator, is the template every conditional-VAE method optimises — the species label is absorbed into the decoder so that the latent $\mathbf{z}$ is, in principle, species-invariant.

Subsetting to one-to-one orthologs is acceptable within mammals (~90 Mya divergence) but discards the majority of genes between vertebrates and invertebrates, and is undefined when no orthology table exists. SATmap [10] replaced the hard ortholog filter with a soft, BLAST-derived gene-homology weight matrix $\mathbf{H}$ and iteratively refined it against cell-level manifold alignment, scoring each cross-species cell pair as

$$M_{ij} = \frac{(\mathbf{x}_i, \mathbf{H} \mathbf{y}_j)}{\|\mathbf{x}_i\| \|\mathbf{H} \mathbf{y}_j\|} \quad (2)$$

which made integration across >500 Mya (sponge-planarian-schistosome-zebrafish) tractable for the first time. SATURN [11] and UCE [12] took the next step: they replace $\mathbf{H}$ entirely with ESM2 protein-language-model embeddings, so that genes from any species are placed in a common 1,280-dimensional protein space *a priori* and no orthology call is ever made.

1 Problem statement

Comparative single-cell transcriptomics asks whether a cell type in one species has a homolog in another, and how its expression program has been conserved or rewired. The technical obstacle is that any two species' transcriptomes live in different gene coordinate systems: orthologs are many-to-many, lineage-specific genes have no counterpart, and even one-to-one orthologs diverge in expression. The first generation of integration tools—Seurat v3's CCA-anchor scheme [1], LIGER's integrative NMF [2], Harmony's iterative clustering correction [3], Scanorama [4] and BBKNN [5]—treat species as a batch covariate after subsetting both datasets to a shared one-to-one ortholog table. Deep generative models followed the same recipe: scVI [6] fits a conditional variational

2 Ortholog-subsetting methods

Seurat v3 [1] finds canonical correlation vectors $\mathbf{u}, \mathbf{v}$ maximising $\text{corr}(\mathbf{u}^T \mathbf{X}, \mathbf{v}^T \mathbf{Y})$ over the shared-ortholog matrices $\mathbf{X}, \mathbf{Y}$, then anchors mutual nearest neighbours in CCA space and applies a smooth correction. LIGER [2] factorises each dataset as $\mathbf{X}_i \approx (\mathbf{W} + \mathbf{V}_i) \mathbf{H}_i$, sharing $\mathbf{W}$ across species and isolating species-specific signal in $\mathbf{V}_i$. Harmony [3] operates post-PCA, iteratively soft-clustering and shifting centroids to maximise batch diversity within clusters; its linear correction is fast but cannot recover signal lost to ortholog subsetting. Scanorama [4] and BBKNN [5] are graph-level: they stitch or rebalance nearest-neighbour graphs across batches without an

Cedars Sinai

31

Thank you for
joining us today.

Share Your Feedback

Scan the QR code to let us know your thoughts on this tutorial.



Everyone who completes the survey will be entered into a drawing to receive a discount code for up to \$250 off an ISCB conference of their choice.

Thank you!